

## Resumen

La creación de APIs RESTful con Node.js es una habilidad fundamental para cualquier desarrollador backend moderno. Aprender a estructurar correctamente tu proyecto y manejar operaciones CRUD (Create, Read, Update, Delete) te permitirá construir aplicaciones robustas y escalables. En esta ocasión, nos enfocaremos en implementar el método GET para recuperar datos de usuarios desde un archivo local, sentando las bases para una futura migración a bases de datos.

¿Cómo configurar un proyecto Node.js para trabajar con archivos locales?

Antes de implementar cualquier endpoint en nuestra API, necesitamos configurar nuestro proyecto para que pueda leer y escribir archivos localmente. Node.js proporciona módulos nativos que facilitan estas operaciones.

Para comenzar, debemos importar los módulos necesarios:

```
const fs = require('fs');
const path = require('path');
```

El módulo `fs` (File System) nos permite trabajar con el sistema de archivos, mientras que `path` nos ayuda a manejar las rutas de los archivos de manera eficiente.

A continuación, necesitamos crear un archivo JSON que funcionará como nuestra "base de datos" temporal:

1. Crea un archivo llamado `users.json` en la raíz del proyecto
2. Define la ruta a este archivo usando el módulo `path`:

```
const usersFilePath = path.join(__dirname, 'users.json');
```

3. Estructura el archivo JSON con algunos datos de prueba:

```
[
  {
    "id": 1,
    "name": "Usuario 1",
    "email": "usuario1@example.com"
  },
  {
    "id": 2,
    "name": "Usuario 2",
    "email": "usuario2@example.com"
  }
]
```

Esta configuración nos permitirá acceder y manipular los datos de usuarios almacenados localmente, **simulando las operaciones que realizaríamos con una base de datos real.**

¿Cómo implementar el método GET para obtener usuarios?

Una vez configurado nuestro proyecto para trabajar con archivos locales, podemos implementar nuestro primer endpoint: GET /users.

```
app.get('/users', (req, res) => {
  fs.readFile(usersFilePath, 'utf8', (error, data) => {
    if (error) {
      return res.status(500).json({
        error: "Error con la conexión de datos"
      });
    }

    const users = JSON.parse(data);
    res.json(users);
  });
});
```

Analicemos este código paso a paso:

1. Utilizamos `app.get()` para definir una ruta que responde a solicitudes HTTP GET
2. La ruta `/users` será el punto de entrada para obtener la lista de usuarios
3. Dentro del callback, usamos `fs.readFile()` para leer el contenido del archivo JSON
4. Incluimos manejo de errores para responder adecuadamente si ocurre algún problema
5. Si todo funciona correctamente, parseamos los datos JSON y los enviamos como respuesta

**Es crucial implementar un manejo adecuado de errores** para proporcionar información útil al cliente cuando algo sale mal. En este caso, devolvemos un código de estado 500 (Error del servidor) junto con un mensaje descriptivo.

Probando nuestro endpoint

Para verificar que nuestro endpoint funciona correctamente, podemos utilizar diferentes herramientas:

1. **Navegador web:** Al ser una solicitud GET, podemos simplemente navegar a `http://localhost:PUERTO/users`
2. **Postman:** Una herramienta más robusta que nos permite documentar y organizar nuestras solicitudes API

En Postman, podemos:

- Crear una nueva colección (por ejemplo, "API Users")
- Añadir una nueva solicitud GET a la URL de nuestro endpoint
- Ejecutar la solicitud y verificar la respuesta

- Guardar la solicitud para uso futuro

**La ventaja de usar Postman es que podemos organizar todas nuestras solicitudes API** y compartirlas fácilmente con otros miembros del equipo, lo que facilita la colaboración y las pruebas.

¿Por qué es importante estructurar correctamente tu API?

La estructura que estamos implementando ahora, aunque utiliza archivos locales, está diseñada para facilitar la transición a una base de datos en el futuro. La lógica básica permanecerá similar:

1. Recibir una solicitud HTTP
2. Obtener datos (ya sea de un archivo local o una base de datos)
3. Procesar esos datos según sea necesario
4. Enviar una respuesta apropiada

Esta separación de preocupaciones nos permite **modificar la fuente de datos sin cambiar significativamente la lógica de nuestra API**. Cuando estemos listos para migrar a una base de datos, solo necesitaremos reemplazar las operaciones de archivo con consultas a la base de datos.

Además, al organizar nuestro código de esta manera, estamos siguiendo buenas prácticas de desarrollo que hacen que nuestro código sea más mantenible y escalable a largo plazo.

La implementación del método GET es solo el primer paso en la creación de una API RESTful completa. En las próximas secciones, exploraremos cómo implementar otros métodos HTTP como POST para crear nuevos usuarios en nuestra "base de datos" local.

¿Has implementado APIs RESTful antes? ¿Qué desafíos has enfrentado al trabajar con Node.js y Express? Comparte tus experiencias en los comentarios y continúa aprendiendo sobre el desarrollo backend.